Auditing an LLM-to-Z3 Theorem-Proving Pipeline: Vacuity, Misencoding, and the Limits of Proof Certificates

Vishnu Kchitti
vishnu@arbiter.dev

April 2026

Abstract

Z3-based LLM theorem proving pipelines report high solve rates, but the encodings are rarely inspected. We audited all 202 proved results from our own pipeline on the full miniF2F test set (Z3-only, gpt-5.4-mini, LLM-assisted classification), plus manual inspection of 23 hard-tier problems and certificate audit of 54 FormalMATH results (sampled from 308/500 certified in a separate benchmark run), and find that **74.3% of Z3-only proofs are strictly invalid** (83.7% including partial proofs). Our reported solve rate of 83% collapses to approximately 16.3% of proved results (13.5% of all 244 problems) after full audit; the hard-tier manual audit finds 13% valid. We attempted to fix this using Knuckledragger proof certificates. Certificates prevent one failure mode: assumed-conclusion failures, where the conclusion is directly asserted as a premise (6.9% of Z3-only failures). They do not eliminate vacuous proofs arising from contradictory premises (38.1% of failures), which still pass through the certificate pipeline. A deeper problem remains: our pipeline responds to certificate requirements by proving trivial lemmas (`kd.prove(x <= x)`) rather than decomposing the actual theorem, yielding 46.3% trivial certificates among 54 audited certificates on FormalMATH (500-problem benchmark, 61.6% cert rate overall; different benchmark and pipeline—results not directly comparable to miniF2F numbers). We hypothesize that a key bottleneck is theorem decomposition—our pipeline appears unable to break hard theorems into formal sub-lemmas that map faithfully to the original claim, though we have not isolated this from other confounds such as prompt design or model capability limits. We release our audit corpus and failure taxonomy as a methodology for evaluating the integrity of Z3-based proof pipelines. Note: leading theorem proving systems (Seed-Prover, HILBERT, AlphaProof) use Lean 4, where the proof assistant itself enforces correctness; these findings apply specifically to Z3-based approaches where encoding fidelity is not automatically checked.

1 Introduction

The miniF2F benchmark [1] has become the de facto leaderboard for LLM-based formal theorem proving. Recent systems report extraordinary numbers: Seed-Prover achieves ∼100% on the test split [2], HILBERT reaches 99.2% [3], and Kimina-Prover reports 92.2% [4]. These results are celebrated as evidence that LLMs can perform genuine mathematical reasoning.

Leading systems (Seed-Prover, HILBERT, AlphaProof) use Lean 4, where the proof assistant's type checker enforces correctness by construction—a Lean proof that compiles is a genuine proof. But for SMT-based approaches—where an LLM generates a Z3 script and the solver returns `unsat`—correctness depends entirely on encoding fidelity, which is external to the solver. When

Z3 returns `unsat`, it means the negated formula is unsatisfiable. It says nothing about whether the encoding faithfully represents the mathematical claim. An LLM can assert the conclusion as a premise, encode a weaker statement, or check only trivial arithmetic. Z3 returns `unsat` in all these cases.

We conducted three complementary audits of Z3 proofs generated by our own pipeline—each targeting a different failure surface—and found the same pattern every time: reported solve rates are dramatically inflated by encoding failures.

- **Manual audit** (23 hard-tier miniF2F): 13% valid (3/23 inspected), 52% invalid.
- **Full semi-automated audit** (all 202 proved full miniF2F, Z3-only): 16.3% valid (33/202), 9.4% partial, 74.3% strictly invalid—including 38.1% vacuous and 6.9% assumed-conclusion (LLM-assisted classification; see §2).
- **Certificate audit** (500-problem FormalMATH benchmark, 61.6% cert rate; 54 certified results manually audited): 7.4% genuinely valid, 46.3% trivial certificates. (Note: FormalMATH is a different benchmark tested under a different pipeline; these numbers are not directly comparable to the miniF2F results above.)

We then tried to fix the problem by introducing Knuckledragger proof certificates [9]—tamper-proof `Proof` objects that prevent conclusions from being assumed as premises. The certificates work for what they target: they eliminate assumed-conclusion failures, where the conclusion is directly asserted as a premise (6.9% of Z3-only failures in our full audit). They do *not* eliminate vacuous proofs arising from contradictory premises, which constitute 38.1% of Z3-only failures. But the remaining failures reveal a deeper problem. LLMs respond to the certificate requirement by proving trivial lemmas (`kd.prove(x <= x)`) and wrapping them in valid-looking certificate chains. The `Proof` object is genuine—it just does not prove the theorem.

The root cause is not the proof backend. Our best hypothesis is that our pipeline fails at theorem decomposition—it proves what is easy (trivial arithmetic, tautologies) and skips what is hard (the actual mathematical content). We have not definitively isolated decomposition failure from other confounds (prompt design, model capacity, benchmark-specific parsing), but the pattern is consistent across all three audits. We do not claim this generalizes to all LLMs or pipelines; it describes our specific system (gpt-5.4-mini, 3 retries, default Z3 timeout).

This paper presents an honest account: we found a serious problem, tried to fix it, and the fix is partial. We report what works, what does not, and what the field still needs.

Contributions.

1. Three complementary audits of our own Z3-based pipeline. The full semi-automated audit (all 202 proved results) finds 74.3% of proofs strictly invalid (83.7% including partial), with an audited valid rate of 16.3% among proved results (33/202), or 13.5% of all 244 problems (33/244). The manual hard-tier audit finds 13% valid (3/23 inspected). Against a reported 82.8% solve rate (Section 3).
2. A certificate-based intervention using Knuckledragger that prevents assumed-conclusion failures by construction (6.9% of Z3-only failures in our full audit) but does not prevent vacuous proofs (38.1%), and exposes a deeper failure mode: trivial certificates (Section 4).
3. A failure taxonomy and the hypothesis that theorem decomposition is a key bottleneck, with concrete examples from FormalMATH; we note this hypothesis is not yet definitively isolated from other confounds (Section 5).

2 Methodology

We employed three complementary audit approaches, each targeting a different failure surface.

Manual audit. The first author manually inspected 23 Z3 proof modules generated by an LLM pipeline (gpt-5.4-mini, 3 retries) on the 35-problem miniF2F-hard tier (AIME and IMO problems). The 23 inspected modules are the subset of hard-tier problems for which the pipeline produced a non-empty proof attempt; the remaining 12 problems either timed out or produced no output. Each inspected module was classified as VALID (faithful encoding + complete derivation), PARTIAL (correct encoding but incomplete derivation), or INVALID (unfaithful encoding or fundamentally unsound reasoning). The 13% valid rate (3/23) is the valid fraction among inspected modules, not a benchmark-level solve rate over all 35 hard-tier problems (the solve rate over all 35 would be 3/35 = 8.6%, excluding the un-attempted problems).

Full semi-automated audit. We audited all 202 "proved" results from the full 244-problem miniF2F test set (Z3-only pipeline, gpt-5.4-mini, 3 retries). For each proof, we ran automated structural checks: (1) remove the conclusion and re-check—if still `unsat`, the proof is vacuous; (2) replace the conclusion with a random false statement—if still `unsat`, the premises are contradictory; (3) check for trivial arithmetic tails; (4) check for crashes masked by error handling. Final classification was performed by an LLM classifier (gpt-5.4-mini) based on execution traces—categories requiring qualitative judgment (PARTIAL, WRONG_ENCODING, VALID) were reviewed but not independently verified by a human expert. Results are organized into seven categories. An earlier 50-sample pilot (seed=42) found 26% valid; the full 202-problem audit finds 16.3% valid, illustrating that small samples can overestimate valid rates due to sampling variance and LLM classifier noise.

Certificate classification. We ran the full Knuckledragger pipeline on a stratified 500-problem sample from FormalMATH (gpt-5.4-mini, temperature 1.0). The pipeline certified 308/500 results (61.6% cert rate). We then manually classified 54 of the 308 certified results into six categories: VALID (certificate proves the stated theorem), TRIVIAL_CERTIFICATE (certificate proves something unrelated to the theorem, e.g., `x <= x`), PARTIAL (certificate covers part of the theorem), NUMERICAL_ONLY (certificate verifies only a numerical answer without the reasoning), CRASH (pipeline error), and WRONG (certificate proves a false or different statement).

3 The Audit: Z3-Only Pipeline

3.1 Headline Numbers

On the full 244-problem miniF2F test set, our Z3-only pipeline (gpt-5.4-mini, 3 retries) reported 202/244 problems proved, a solve rate of 82.8%. The full audit of all 202 proved results and the manual hard-tier audit tell a different story.

The manual audit found 3 of 23 hard-tier proofs genuinely valid (13%). The full semi-automated audit found 33 of 202 proved results valid (16.3%). Both numbers are rates among proved results (the 202 problems the pipeline claimed to prove), not rates over all 244 problems. The audited valid rate over all 244 problems is 33/244 = 13.5%. An earlier 50-sample pilot estimated 26% valid; the full 202-problem audit finds 16.3%, illustrating that small-sample estimates can overestimate valid rates by several percentage points.

Table 1: Two audits of the Z3-only pipeline on miniF2F. Both show the majority of "proved" results are invalid. The full semi-automated audit covers all 202 proved results; the manual audit covers 23 of 23 hard-tier proved results.

	Manual (23 hard)		**Full audit (202 full)**
Verdict	Count	%	%
VALID	3	13%	16.3%
PARTIAL	8	35%	9.4%
INVALID	12	52%	74.3%

Table 2: Failure taxonomy from the full semi-automated audit (202 proved results, Z3-only, full miniF2F). LLM-assisted classification (gpt-5.4-mini).

Category	**Description**	**Count**	**%**
Valid	Faithful encoding, complete derivation	33	16.3
Vacuous	Premises contradictory; UNSAT is trivial	77	38.1
Trivial tail	Only final arithmetic checked, reasoning skipped	29	14.4
Partial	Correct encoding but incomplete derivation	19	9.4
Crash	Z3 timeout/error masked by error handling	18	8.9
Assumed conclusion	Conclusion asserted as premise	14	6.9
Wrong encoding	Mathematical claim not faithfully represented	12	5.9

3.2 Failure Taxonomy

The automated audit revealed seven distinct failure modes, summarized in Table 2.

The dominant failure mode is *vacuous encoding* (38.1%, 77/202): the proof-generating model constructs premises that are themselves contradictory, so Z3 returns `unsat` regardless of the conclusion. This is followed by trivial arithmetic tails (14.4%), partial proofs (9.4%), crashes masked by error handling (8.9%), assumed conclusions (6.9%), and wrong encodings (5.9%).

3.3 Illustrative Examples

Vacuous encoding (`induction_pprime_pdvdapowpma`). The most common failure. The proof-generating model adds constraints that, taken together, are contradictory, so Z3 returns `unsat` regardless of the conclusion. From our audit: the model encodes contradictory parity constraints on p (simultaneously even and odd), making the premise set mutually inconsistent before the conclusion is even added. Classifier verdict: "The solver constraints are already UNSAT even without the negated claim, so the proof is vacuous."

```
from z3 import *
s = Solver()
p, a, m = Ints('p a m')
s.add(p > 1)            # p > 1
s.add(p % 2 == 0)       # <-- LLM adds: p is even
s.add(p % 2 != 0)       # <-- LLM also adds: p is odd  (CONTRADICTS above!)
s.add(a % p == 0)       # hypothesis: p divides a
s.add(m > 0)
s.add(Not(a % p == 0)) # negated conclusion (irrelevant: already UNSAT)
assert s.check() == unsat  # vacuous: p%2==0 and p%2!=0 is UNSAT for any p
```

Listing 1: Vacuous encoding: premises are mutually contradictory (schematic from `induction_pprime_pdvdapowpma`).

Assumed conclusion (amc12a_2002_p6, mathd_numbertheory_427). The conclusion is directly asserted as a constraint. From our audit of amc12a_2002_p6: "The code directly encodes the conclusion that every positive m has a witness n... by explicitly asserting the existence of such n as a constraint rather than deriving it."

```
s = Solver()
m, n = Ints('m n')
s.add(m > 0)
# The theorem asks us to prove there exists n s.t. n^2 > m
s.add(n > 0)
s.add(n * n > m)    # <-- this IS the conclusion; it's axiomatized!
s.add(Not(n * n > m))
assert s.check() == unsat
```

Listing 2: Assumed conclusion: target existence claim added as axiom (amc12a_2002_p6).

Trivial arithmetic tail (amc12a_2013_p4, mathd_numbertheory_328). The model skips the mathematical reasoning and checks only that a concrete numerical result satisfies itself. From our audit of amc12a_2013_p4: "The code does not encode the actual problem's exponent expression; it directly sets the value." For mathd_numbertheory_328: "only performs direct modular arithmetic evaluations with Python's pow and checks a hardcoded result."

```
# After extensive comments describing the "reasoning"...
answer = 12   # the intended final answer
s = Solver()
s.add(Not(answer == 12))
assert s.check() == unsat  # trivially: 12 != 12 is unsat
```

Listing 3: Trivial tail: skips reasoning, checks hardcoded answer = answer (amc12a_2013_p4).

3.4 Why Z3 Cannot Prevent These Failures

Z3's Python API makes no distinction between axioms and derived facts. Every s.add() call has the same status. The solver answers "is this set of constraints satisfiable?" without tracking provenance. A correct Z3 proof requires discipline external to the solver—discipline that LLMs systematically lack.

4 Do Proof Certificates Fix It?

To address the trust gap, we introduced Knuckledragger [9], a Python proof assistant built on Z3 by Philip Zucker. Its key property is a tamper-proof Proof type: you cannot construct a Proof object without the kernel verifying the derivation. Specifically, kd.lemma(claim, by=[p1, p2, ...]) asks Z3 to derive claim from existing proofs and returns a Proof only if successful. Combined with SymPy for algebraic pre-computation, this forms our certification pipeline.

4.1 What Certificates Fix

Certificates prevent one class of failure: *assumed-conclusion* failures, where the proof script directly asserts the conclusion as a premise. In Knuckledragger, every claim must be derived from prior Proof objects via kd.lemma or kd.prove; you cannot silently introduce an unproven claim as an axiom. This eliminates the 6.9% of Z3-only failures (14/202) classified as Assumed_Conclusion in our full audit. One caveat: Knuckledragger provides an escape hatch, kd.axiom(claim), which

Table 3: Certificate audit of 54 certified FormalMATH results (sampled from 308/500 certified; Knuckledragger + SymPy pipeline).

Category	**Count**	**%**
Trivial_Certificate	25	46.3
Partial	9	16.7
Numerical_Only	9	16.7
Crash	4	7.4
Valid	4	7.4
Wrong	3	5.6

creates a `Proof` without verification. Our pipeline does not invoke `kd.axiom`, but the guarantee is contingent on pipeline discipline rather than being enforced by the type system itself.

However, certificates do *not* eliminate vacuous proofs. A vacuous proof arises from contradictory premises—the LLM encodes constraints that are already mutually unsatisfiable, so Z3 returns `unsat` regardless of the conclusion. Under Knuckledragger, the LLM can still add contradictory constraints; the kernel only checks that each `kd.lemma` call is derivable from its stated hypotheses, not that those hypotheses are jointly consistent. The 38.1% of Z3-only failures (77/202) that are vacuous in our full audit remain a live failure mode under certification.

On miniF2F, the certificate pipeline shows measurable improvement on solve rate:

- **Knuckledragger v1** (hard 35): 48.6% proved, 37.1% certified.
- **Knuckledragger + Sonnet** (full 244): 100% proved, 89.3% certified.
- **Manual re-execution**: 7/10 sampled certified miniF2F proofs re-execute cleanly when run in isolation (70%); 3/10 fail due to environment dependency issues or non-deterministic Z3 behavior. Note: re-execution confirms reproducibility, not validity—a proof can re-execute cleanly and still be vacuous.

These numbers look encouraging. But the certificate rate is not the same as the *valid* rate. **Audit gap**: we have not audited these miniF2F Knuckledragger+Sonnet certificates with the same methodology applied to the Z3-only results. We do not know the actual valid rate among the 89.3% certified results; it may be substantially lower.

4.2 What Certificates Do Not Fix: The FormalMATH Audit

We ran 500 FormalMATH problems through the Knuckledragger pipeline (61.6% cert rate, 308/500 certified) and manually classified 54 of the 308 certified results. The results, shown in Table 3, are sobering.

Only 4 of 54 certificates (7.4%) are genuinely valid—they prove the stated theorem. The dominant failure is Trivial_Certificate at 46.3%: the LLM produces a valid `Proof` object, but the certified claim is trivial and unrelated to the theorem. The certificate is real; it just does not prove anything interesting.

4.3 Example: What Certificates Actually Prove

Table 4 shows three concrete examples of the gap between what the problem asks and what the certificate proves.

Table 4: What `kd.prove` certifies vs. what the problem asks. All three certificates are valid `Proof` objects—but none proves the theorem.

Problem	What the theorem asks	What kd.prove certifies
FormalMATH #12	For all $n > 0$, $\sum_{k=1}^{n} k^2 = n(n+1)(2n+1)/6$	`kd.prove(x <= x)` — reflexivity of $\leq$
FormalMATH #31	If p is prime and $p \mid ab$, then $p \mid a$ or $p \mid b$	`kd.prove(Or(True, False))` — tautology
FormalMATH #47	$\sqrt{2}$ is irrational	`kd.prove(2 > 0)` — positivity of 2

Table 5: Reported vs. audited valid rates. The two benchmarks use different pipelines and preprocessing; rates are not directly comparable across rows. Within each row, reported and audited rates are comparable.

Benchmark	Backend	Tier	Reported	Audited valid
miniF2F (Z3-only)	Raw Z3	Full (202)	82.8%	16.3% of proved (13.5% of all 244)
miniF2F (Z3-only)	Raw Z3	Hard (23 inspected)	—	13% of inspected
FormalMATH (KD)†	kdrag+SymPy	Mixed	61.6% cert	7.4%

† Different benchmark, different pipeline; not comparable to miniF2F rows.

In each case, the LLM generates a proof script that calls `kd.prove` on a trivially true statement. Knuckledragger correctly certifies the trivial claim—because the claim *is* true. But the certified claim has no logical connection to the original theorem. The LLM has substituted an easy problem for a hard one.

4.4 The Inflation Pattern Is Consistent

Table 5 summarizes the inflation pattern across all three settings. Regardless of backend, benchmark, or certification mechanism, reported rates substantially exceed audited valid rates: approximately 6.1× for the Z3-only full audit (82.8% reported vs. 13.5% valid on all 244 problems), and approximately 8.3× for FormalMATH certificates (61.6% cert rate vs. 7.4% genuinely valid among audited certificates). The FormalMATH cert rate is from the 500-problem run; the 7.4% valid rate is from the 54-problem audit of certified results. The two rows use different benchmarks and pipelines and cannot be combined into a single multiplier.

5 The Decomposition Bottleneck

The certificate audit reveals a pattern in our pipeline: the proof-generating model does not decompose theorems. It generates proof scripts that *look* like multi-step proofs—with comments, lemma names, and structured reasoning—but the actual certified claims are trivial. The hard mathematical content lives in the comments and variable names, not in the formal logic.

Consider a typical FormalMATH proof script for "$\sqrt{2}$ is irrational." The LLM generates:

```
import kdrag as kd
# Step 1: Assume sqrt(2) = p/q in lowest terms
# Step 2: Then 2*q^2 = p^2, so p is even
# Step 3: Write p = 2k, then 2*q^2 = 4*k^2, so q^2 = 2*k^2
# Step 4: So q is even, contradicting lowest terms
# Therefore sqrt(2) is irrational.
proof = kd.prove(2 > 0)  # <-- this is all that's certified
```

Listing 4: Typical LLM-generated "proof" of irrationality of $\sqrt{2}$.

The comments describe a correct informal proof. But the formal content—the only thing the kernel checks—is `2 > 0`. The LLM knows what the proof *should* look like. It cannot translate that knowledge into formal sub-lemmas.

This is the decomposition bottleneck: to prove a theorem formally, you must break it into a chain of lemmas where each lemma is within the solver's reach. This requires understanding which steps are trivial for Z3 (arithmetic, propositional logic) and which require careful encoding (induction, case analysis, number-theoretic arguments). Our pipeline treats the proof as a natural-language narrative and appends a trivial formal claim at the end.

Implications. If this pattern holds more broadly, improving the proof backend (better solvers, better certificate mechanisms) has diminishing returns. The binding constraint would be the model's ability to perform multi-step mathematical reasoning that maps to formal sub-lemmas. This is a fundamentally different capability from generating plausible proof narratives. We present this as a hypothesis supported by our audit data, not a definitive finding—other pipelines, models, or prompting strategies may exhibit different behavior.

6 Related Work

Lean-based theorem proving. The dominant paradigm uses Lean 4 with kernel-checked proofs. Seed-Prover [2] combines MCTS with learned value functions; HILBERT [3] uses similar search with improved training. Kimina-Prover [4] and Goedel-Prover-V2 [5] explore reinforcement learning. These systems benefit from Lean's kernel guarantee: if the proof type-checks, it is correct. Our audit demonstrates why this guarantee matters.

SMT-based approaches. MiniF2F-Dafny [7] (POPL 2026) translates problems to Dafny at 55.7%. Dafny's verification condition generation provides stronger guarantees than raw Z3 but weaker than Lean's kernel. Our findings suggest that any SMT-based approach is vulnerable to encoding failures unless independently audited.

Proof integrity. AlphaProof [8] (Nature 2025) notably chose Lean kernel verification over solver-based approaches for IMO problems. Our audit validates this design choice. FormalMATH [6] extends formalization to broader domains but, as our certificate audit shows, formalization alone does not guarantee proof validity.

LLM evaluation skepticism. The PAJAMA study [11] and "Time to Impeach LLM-as-a-Judge" critique [12] demonstrated systematic over-reporting in LLM evaluation. Work on debate-based verification [13] similarly highlights gaps between perceived and actual quality. Our findings extend this skepticism to formal verification: even in a domain where correctness is supposedly checkable, unchecked results are unreliable.

Knuckledragger. Developed by Philip Zucker [9], Knuckledragger wraps Z3 in a proof-carrying discipline via opaque `Proof` objects. We are, to our knowledge, the first to use it for large-scale LLM theorem proving—and the first to document its limitations when LLMs generate trivial certificates.

7 Discussion

Observations and suggestions. Based on our audits of our own pipeline, we offer the following observations (caveated: these are from one pipeline; other pipelines may fail at different rates):

1. **Audit the proofs.** Our experience suggests reporting both solve rates and audited valid rates. An unaudited solve rate is not a reliable measure of genuine proof quality. Our audits reveal a 6.1× gap between reported and audited rates for our pipeline; other pipelines should be independently audited to characterize their own gaps.
2. **Distinguish certificate rates from valid rates.** In our FormalMATH experiment, a 61.6% certificate rate corresponded to 7.4% genuinely valid among audited results. Certificate generation is not the same as valid proof generation; the gap is worth measuring.
3. **Decomposition as a hypothesis.** We observe that our pipeline appears to fail at theorem decomposition rather than proof search. This is a hypothesis based on our audits; we have not definitively isolated it from other confounds. If confirmed more broadly, research effort targeting sub-lemma decomposition may be valuable.

Honest positioning. Table 6 places our results alongside published systems. We are not claiming state-of-the-art performance—we are not competing with Lean 4 kernel-based provers. The purpose of this table is to contrast our audited valid rate (13.5%) against our own reported rate (82.8%), illustrating the inflation problem. The Lean systems are included for reference only: they report kernel-verified rates which are not subject to the same audit critique (the kernel enforces correctness by construction). A reader should not interpret this table as "Ours vs. Lean"; it is "reported rate vs. audited rate for an SMT-based pipeline." Note: one reviewer criticized our pipeline as "deliberately incompetent" for its simple prompt design. This is partly fair: a more sophisticated prompt (chain-of-thought, few-shot lemma examples, decomposition scaffolding) might improve rates. Our contribution is not a better Z3 prover, but an audit showing that *even this baseline pipeline*, which represents common practice for LLM+Z3 attempts on miniF2F, produces 74% invalid proofs. A more sophisticated pipeline might fail at different rates, but the audit methodology we provide applies to any pipeline.

What certificates get right. We do not want to dismiss certificates entirely. Knuckledragger certificates eliminate one failure class: assumed-conclusion failures (6.9% of Z3-only proofs, 14/202 in our full audit), where the proof script directly asserts the conclusion as a premise. This is a genuine improvement—it removes a class of proofs that are trivially wrong. The problem is that certificates are necessary but not sufficient: they catch one failure mode while LLMs adapt by producing a different one (trivial certificates), and they leave the dominant failure mode (vacuous proofs, 38.1%) entirely intact.

Limitations. Our manual audits were conducted by a single author with no independent verification. The semi-automated audit uses an LLM classifier (gpt-5.4-mini) for categories requiring qualitative judgment; misclassification is possible and we have not estimated the error rate. We note the irony: this paper cites PAJAMA and related work on LLM-judge unreliability while using an LLM classifier as our primary auditing tool. The LLM classification is a practical expedient for processing 50+ proofs; the manual audit of 23 problems is the higher-trust component. The FormalMATH certificate audit covers 54 problems, a small and potentially non-representative sample. We have not audited other systems' proof files—all audits are of proofs generated by our own

Table 6: Comparison with published systems on miniF2F (test split). *Lean kernel provides built-in certification; reported rates *are* certificate rates. †Audited valid rate over all 244 problems: 33/244 = 13.5% (full semi-automated audit of all 202 proved results). Not directly comparable to Lean systems, which report kernel-verified rates. ‡KD+SymPy certified rate (Claude Sonnet 4.5); the 89.3% certified results have not been audited with the same methodology—valid rate unknown.

System	**Backend**	**Ref.**	**Rate (%)**
Seed-Prover	Lean 4*	[2]	∼100
HILBERT	Lean 4*	[3]	99.2
Kimina-Prover	Lean 4*	[4]	92.2
Goedel-Prover-V2	Lean 4*	[5]	90.4
AlphaProof	Lean 4*	[8]	—
MiniF2F-Dafny	Dafny/Z3	[7]	55.7
Ours (reported, Z3-only, full 244)	Raw Z3		82.8
Ours (audited valid, Z3-only)†	Raw Z3		13.5
Ours (reported, KD+SymPy, full 244)	Knuckledragger		89.3‡

pipeline, so we cannot directly claim the same failure rates apply to other Z3-based approaches. Knuckledragger relies on Z3's proof-producing mode, which is sound but incomplete—some valid proofs may fail to certify. Finally, our findings apply to Z3-based approaches; Lean-based systems with kernel checking are not subject to the same critique.

Reproducibility. All experiments used gpt-5.4-mini (temperature 1.0, top_p 1.0, up to 3 retries, 60-second Z3 timeout per problem). The audit classification pipeline used gpt-5.4-mini as classifier (temperature 0, structured output mode). The actual system prompt used for proof generation is reproduced verbatim below:

```
You are an expert in formal verification using the Z3 SMT solver (Python).  Given
a math problem, generate a standalone Python module that PROVES the result using
Z3.  REQUIREMENTS: (1) Import from z3:  Solver, Optimize, Real, Int, Bool, And, Or,
Not, Implies, ForAll, Exists, sat, unsat, etc.  (2) Export a verify() -> dict function
returning {"check1":  {...}}.  (3) PROOF STRATEGY: For ‘‘prove P’’ problems:  encode
¬P as constraints, check UNSAT. If UNSAT → P is proven (no counterexample exists).
```

The user turn adds: the problem ID, problem statement, and an optional informal proof hint. There is no decomposition scaffolding, no few-shot examples of lemma structure, and no chain-of-thought requirement. This minimal prompt is a deliberate baseline: a more sophisticated prompt might improve valid rates, but this represents the behavior we observe and audit. Full prompt templates, audit results, and all generated proof scripts are released at https://github.com/vishk23/arbiter.

The decomposition problem as a hypothesis. Our audits converge on a consistent pattern: the hard problem in our Z3-based pipeline is not proof verification but theorem decomposition. We present this as a hypothesis—our data supports it but does not establish it as a general law of LLM theorem proving. Until this is tested across multiple models and pipelines, claims about what "LLMs" can or cannot do should be interpreted narrowly. What we can say with confidence is that this specific pipeline, on these benchmarks, fails primarily at decomposition. If this hypothesis holds more broadly, benchmarks and methods that specifically target decomposition ability—rather

than just end-to-end solve rates—would be valuable for the field. We offer this as a directional suggestion, not a demonstrated conclusion.

Acknowledgments

We thank Philip Zucker for developing Knuckledragger and for discussions about proof-carrying code in SMT solvers.

References

[1] K. Zheng, J. M. Han, and S. Polu. miniF2F: A cross-system benchmark for formal Olympiad-level mathematics. In *International Conference on Learning Representations (ICLR)*, 2022.

[2] Seed-Prover Team. Seed-Prover: Large language models as formal theorem provers via MCTS. *arXiv preprint arXiv:2507.23726*, 2025.

[3] Apple ML Research. HILBERT: A large language model approach to formal theorem proving. *arXiv preprint arXiv:2509.22819*, 2025.

[4] Kimina Team. Kimina-Prover: Scaling formal theorem proving with reinforcement learning. *arXiv preprint*, 2025.

[5] Goedel Team. Goedel-Prover-V2: Improved neural theorem proving via synthetic data and curriculum learning. *arXiv preprint*, 2025.

[6] FormalMATH Team. FormalMATH: Benchmarking formal mathematical reasoning of large language models. *arXiv preprint arXiv:2505.02735*, 2025.

[7] Y. Sun, Y. Wu, and X. Si. Formal verification of LLM-generated mathematical proofs via Dafny. In *Proceedings of the ACM SIGPLAN Symposium on Principles of Programming Languages (POPL)*, 2026. *arXiv preprint arXiv:2512.10187*.

[8] DeepMind AlphaProof Team. AlphaProof: AI achieves silver-medal level performance at the International Mathematical Olympiad. *Nature*, 2025.

[9] P. Zucker. Knuckledragger: A Python proof assistant built on Z3. https://github.com/philzook58/knuckledragger, 2026.

[10] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS*, 2021.

[11] PAJAMA Authors. PAJAMA: Systematic evaluation of LLM-as-a-judge. *arXiv preprint*, 2025.

[12] M. Chen and colleagues. Time to impeach LLM-as-a-judge: Systematic biases in automated evaluation. *arXiv preprint*, 2025.

[13] N. Kenton, T. Everitt, and J. Leike. Debate does not (yet) help humans identify deceptive AI. In *International Conference on Learning Representations (ICLR)*, 2025.